Predictive:
data preprocessing

Data preprocessing

Raw data could be unsuitable for forecasting algorithms:

- Some algorithms require data with specific characteristics
- Some algorithms perform better when data is presented in some specific format.

Data preprocessing has the objective of making raw data better compatible with the selected algorithm.

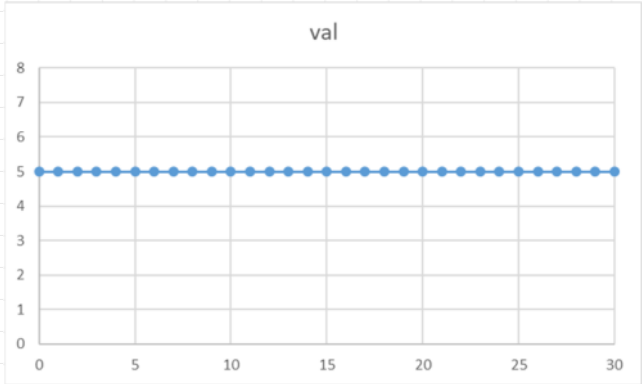
Many different preprocessing options.

No deterministic rule on what to do.

Preprocessing

The easiest time series to forecast.

Data preprocessing tries to revert to this case.



Fundamental moments

A moment is a specific quantitative measure of the shape of a function.

The statistical description of a time series $Y_t = (y_1, \dots, y_n)$, is based on three main moments:

- mean: $\mu = E[Y_t]$;
- variance: $\sigma_t^2 = E[Y_t - \mu]^2$
- autocovariance: $\gamma_{t,s} = E[(Y_t - \mu)(Y_s - \mu)]$ (relates the value at time t with the value at time s. Represented by *autocorrelograms*)

Stationary series

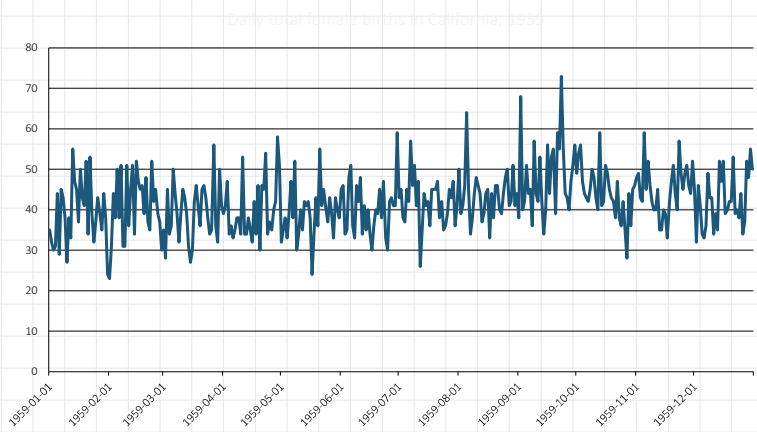
Stationarity conditions. A time series is stationary if:

1. Its mean is constant over time (stationarity in means);
2. Its variance is constant over time (stationarity in variance);
3. The ratio between values separated by k time periods depends only on k, not on time (stationarity in covariance).

Simply, a series is stationary if its distribution is invariant w.r.t. translations in time.

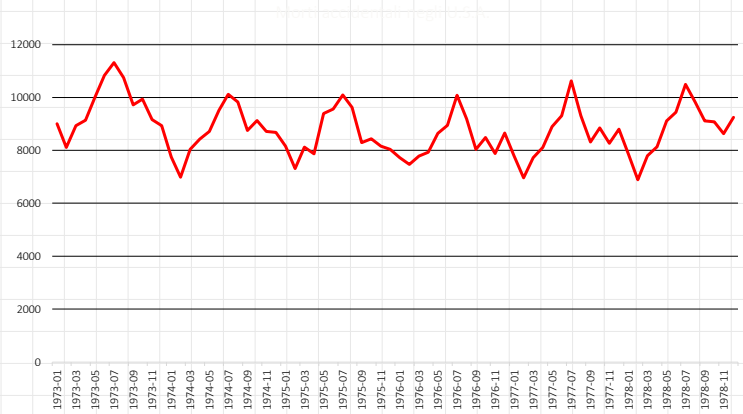
Stationary process

It has no trends (stationary in means and variance). No seasonality or cyclicity.



Stationary with seasonality

Stationary in means and variance, but with seasonality.



Basic transforms

Given a univariate time series dataset, there are some basic transforms often used prior to forecasting:

- Difference Transform
- Power Transform
- Standardization
- Normalization

Difference transform

We can try to make a time series stationary in mean by (first order) differencing, i.e., by subtracting from the current value the value of the previous time moment:

$$y'_t = y_t - y_{t-1}$$

If the series y_t has a linear trend, the resulting series y'_t is stationary in means.

Second order differencing:

$$y''_t = y'_t - y'_{t-1} = y_t - 2y_{t-1} + y_{t-2}$$

Seasonal differencing: subtract from the current value the corresponding one of the previous season:

$$y'_t = y_t - y_{t-m}$$

Difference Transform

Difference transform (*diff*) subtracts from each entry in the series the entry back-shifted of a constant time lag.

This is a simple way for removing a systematic structure from the series: e.g., a linear trend (first order differencing).

The process can be repeated (e.g. difference the differenced series) to remove second order trends, and so on.

A seasonal structure can be removed by subtracting the observation from the prior season, e.g. 12 time steps before for monthly data with a yearly seasonal structure.

Fil rouge, diff transform

t	y_t	diff
1	128	
2	181	53
3	87	-94
4	219	132
5	407	188
6	226	-181
7	214	-12
8	383	169
9	505	122
10	387	-118
11	278	-109
12	523	245
13	572	49
14	354	-218
15	404	50
16	673	269
17	752	79
18	468	-284
19	419	-49
20	725	306

Removed the linear trend!

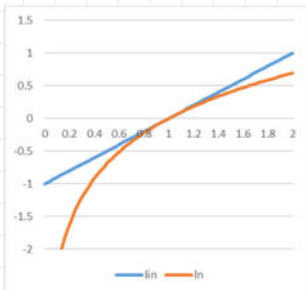
difference transform

```
# example of a difference transform (python)
# difference dataset
def difference(data, interval):
   return [data[i] - data[i-interval] for i in range(interval,len(data))]
# invert difference
def invert_difference(orig_data, diff_data, interval):
   inverted = [diff_data[i]+orig_data[i] for i in range(interval)]
   for i in range(interval,len(diff_data)):
      inverted = np.append(inverted,diff_data[i]+inverted[i-interval])
   return inverted
# simple dataset as example
data = [5+2*x for x in range(0,10)]
print(data)
# difference transform
transformed = difference(data, 1)
print(transformed)
# invert difference
inverted = invert_difference(data[:1], transformed, 1)
inverted = np.append(data[:1],inverted)
print(inverted)
```

t	*series*	*diff*
1	5	
2	7	2
3	9	2
4	11	2
5	13	2
6	15	2
7	17	2
8	19	2
9	21	2
10	23	2

Log transform

We can try to make a series stationary in variance with a log transform (and, with diff, also in mean).



The function $y = ln(x)$ in $(1,0)$ is tangent to the line $y = x-1$

Thus $ln(1+r) \approx r$ when r is small w.r. to 1 (f.e., 1 ± 0.2).

If x has a small percent increase, ex. 5%, it changes from x to $(1+r)x$, with $r = 0.05$. We have that:

$ln((1+r)x) = ln(x) + ln(1+r) \approx ln(x) + r$

If x increases by 5% the log of x increases from $ln(x)$ to $ln(x) + 0.05$.

The increase of x by 5% corresponds (roughly) to an increase of 0.05 to $ln(x)$.

x could be a percent variation of a series y at time t, defined as $(y_t-y_{t-1})/y_{t-1}$

A percent variation, is almost equal to $ln(y_t) - ln(y_{t-1})$

Log transform

Example Airline passengers (first records)

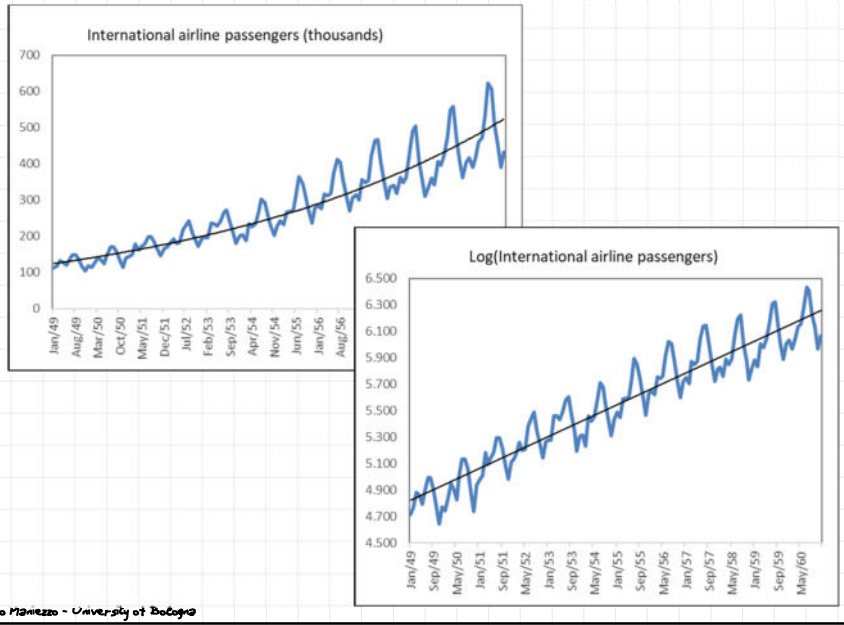
Passengers	% var (r)	ln(x)	ln(x-1)+r
112		4.718499	
118	0.053571	4.770685	4.772070
132	0.118644	4.882802	4.889329
129	-0.02273	4.859812	4.860075
121	-0.06202	4.795791	4.797797
135	0.115702	4.905275	4.911493
148	0.096296	4.997212	5.001571
148	0	4.997212	4.997212
136	-0.08108	4.912655	4.916131
119	-0.125	4.779123	4.787655
104	-0.12605	4.644391	4.653073
118	0.134615	4.770685	4.779006
115	-0.02542	4.744932	4.745261
126	0.095652	4.836282	4.840584
141	0.119048	4.94876	4.95533
135	-0.04255	4.905275	4.906207
125	-0.07407	4.828314	4.831201
149	0.192	5.003946	5.020314
170	0.14094	5.135798	5.144886
170	0	5.135798	5.135798

Log transform

Log transform has the following main properties:

- Linearization of exponential increases: $ln(xy) = ln(x) + ln(y)$, log transform turns multiplicative relations into additive and exponential trends into linear. It makes it possible to use linear models for exponential processes.
- Trends measured in log units ≈ percentual increases: the slope of a trend on a log scale is equal to the percentual increase of the original series.
- Errors measures in log units ≈ percentual errors: errors measured on log data can be seen as percentual errors on original data.

Log transform, example



Fil rouge, log transform

t	y_t	*log*	*log-diff*
1	128	4.85	
2	181	5.20	0.35
3	87	4.47	-0.73
4	219	5.39	0.92
5	407	6.01	0.62
6	226	5.42	-0.59
7	214	5.37	-0.05
8	383	5.95	0.58
9	505	6.22	0.28
10	387	5.96	-0.27
11	278	5.63	-0.33
12	523	6.26	0.63
13	572	6.35	0.09
14	354	5.87	-0.48
15	404	6.00	0.13
16	673	6.51	0.51
17	752	6.62	0.11
18	468	6.15	-0.47
19	419	6.04	-0.11
20	725	6.59	0.55

Removed the trend on variance!

Removed the trend on means and variance!

Power Transform (log)

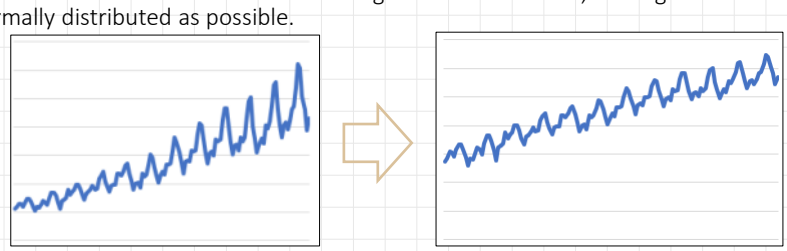
Power transform is a family of functions that create a monotonic transformation of data using power functions, used to stabilize variance, make the data more normal distribution-like.

On a time series dataset, this can remove a change in variance over time.

Popular examples are the log transform, the square root transform or generalized versions such as the Box-Cox transform.

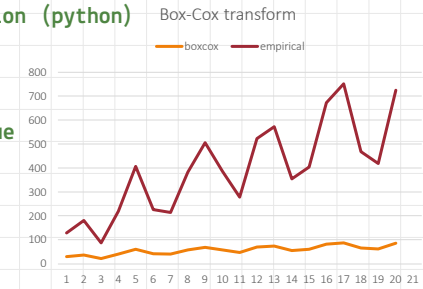
Boxcox transforms positive data according to the formula $y(\lambda) = \begin{cases} \frac{x^{\lambda}-1}{\lambda} & \lambda \neq 0 \\ \ln(x) & \lambda = 0 \end{cases}$

The λ value is chosen to maximize the log-likelihood function, making the data as normally distributed as possible.



Power (Box-Cox) transform

```
# example of power transform and inversion (python)
from math import log,exp
from scipy.stats import boxcox
import pandas as pd
# invert a boxcox transform for one value
def invert_boxcox(value, lam):
        # log case
        if lam == 0:
           return exp(value)
        # general case
        return exp(log(lam * value + 1) / lam)
# define dataset
data = pd.read_csv('FilRouge.csv',usecols=[1]).values.flatten()
print(data)
# power transform
transformed, lmbda = boxcox(data)
print(f"data={transformed}, lmbda={lmbda}")
# invert transform
inverted = [invert_boxcox(x, lmbda) for x in transformed]
print(inverted)
```



Quest for stationarity

A pipeline of successive transformations, each working on the result of the preceding one.

- Log (Box Cox) transform to move from exponential to linear trends and from multiplicative to additive variations (f.e., variance).
- Diff transform, d=1, removes linear trends (need to store the original first value apart).
- Diff transform, d=m, removes seasonality of order m (need to store the original m values apart).

The result *could be* stationary.

All transformations are invertible.

Example (fil rouge)

t	sales	log	diff 1	diff m	reconstruct	reconstruct	reconstruct
1	128	4.8520				4.8520	128
2	181	5.1985	0.3465		0.3465	5.1985	181
3	87	4.4659	-0.7326		-0.7326	4.4659	87
4	219	5.3891	0.9232		0.9232	5.3891	219
5	407	6.0088	0.6197		0.6197	6.0088	407
6	226	5.4205	-0.5883	-0.9347	-0.5883	5.4205	226
7	214	5.3660	-0.0546	0.6780	-0.0546	5.3660	214
8	383	5.9480	0.5821	-0.3411	0.5821	5.9480	383
9	505	6.2246	0.2765	-0.3432	0.2765	6.2246	505
10	387	5.9584	-0.2661	0.3221	-0.2661	5.9584	387
11	278	5.6276	-0.3308	-0.2762	-0.3308	5.6276	278
12	523	6.2596	0.6320	0.0499	0.6320	6.2596	523
13	572	6.3491	0.0896	-0.1870	0.0896	6.3491	572
14	354	5.8693	-0.4798	-0.2137	-0.4798	5.8693	354
15	404	6.0014	0.1321	0.4629	0.1321	6.0014	404
16	673	6.5117	0.5103	-0.1216	0.5103	6.5117	673
17	752	6.6227	0.1110	0.0214	0.1110	6.6227	752
18	468	6.1485	-0.4743	0.0056	-0.4743	6.1485	468
19	419	6.0379	-0.1106	-0.2427	-0.1106	6.0379	419
20	725	6.5862	0.5483	0.0380	0.5483	6.5862	725

Example (fil rouge)

```
df = pd.read_csv('FilRouge.csv', usecols=[1])

# log transform
dflog =  np.log(df['sales'])

# 1 order diff
diff1 = [dflog[i] - dflog[i - 1] for i in range(1, len(dflog))]

# diff of m
m = 4
diffm = [diff1[i] - diff1[i - m] for i in range(m, len(diff1))]

# ... Work on this, they should be stationary ...

# reconstrunct diff1
rec1 = diff1[:m] + diffm
for i in range(m, len(rec1)):
   rec1[i] += rec1[i - m]

# reconstruct dflog
reclog = [dflog[0]] + rec1
for i in range(1, len(rec1)):
   reclog[i] += reclog[i - 1]

# reconstruct original
recdata = np.exp(reclog)
```

Test for stationarity

Augmented Dikey Fuller stationarity test

```
import numpy as np
from statsmodels.tsa.stattools import adfuller

arr = diffm  # Replace with your data of interest
# ADF test
adf_result = adfuller(arr) # default autolag='AIC'

# results (ADF statistic and p-value)
print(f'ADF Statistic: {adf_result[0]}')
print(f'p-value: {adf_result[1]}')
```

p-value ≤ 0.05 → Reject the null hypothesis → The series is stationary.
p-value > 0.05 → Fail to reject the null hypothesis → The series is non-stationary.

Test for normality

Residuals should be normally distributed.

Shapiro-Wilk Test, good for small samples (n < 5000).

```
import numpy as np
from scipy import stats
# Shapiro-Wilk Test
shapiro_test = stats.shapiro(arr)
print(f"Shapiro-Wilk Test: Statistic={shapiro_test.statistic}, p-value={shapiro_test.pvalue}")
```

Example (on diffm):

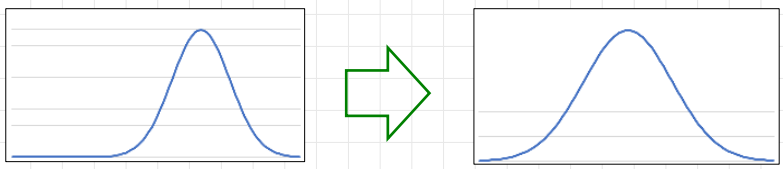
```
Shapiro-Wilk Test: Statistic=0.946992312712177, p-value=0.4783874457749567
```

Standardization

Standardization is a transform for data with a Gaussian distribution resulting in a standard normal.

It rescales the distribution of values so that the mean of observed values is 0 and the standard deviation is 1.

It subtracts the series mean $\bar{x}$ and divides the result by the standard deviation σ of the data sample: $z_i = \frac{x_i - \bar{x}}{\sigma}$



Standardization

```
from sklearn.preprocessing import StandardScaler
# example of standardization
# define data, it must be a column vector
data = pd.read_csv('FilRouge.csv',usecols=[1]).values
print(data.T)
# fit transform
transformer = StandardScaler()
transformer.fit(data)
# data transformation
transformed = transformer.transform(data)
print(transformed.T)
# invert scaling
inverted = transformer.inverse_transform(transformed)
print(inverted.T)
```

```
[[128 181  87 219 407 226 214 383 505 387 278 523 572 354 404 673 752 468
  419 725]]
[[-1.42912421 -1.14570575 -1.64837245 -0.94250006  0.06283334 -0.90506743
  -0.96923765 -0.0655071   0.58689011 -0.04411702 -0.62699649  0.68314543
   0.94517382 -0.22058512  0.04679078  1.48527315  1.90772707  0.38903194
   0.12700355  1.76334409]]
[[128. 181.  87. 219. 407. 226. 214. 383. 505. 387. 278. 523. 572. 354.
  404. 673. 752. 468. 419. 725.]]
```

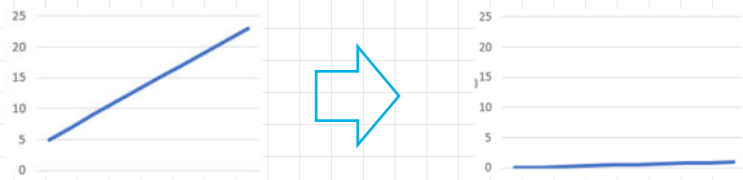
Average = 0
Std.dev. = 1

Normalization

In the simplest case, normalization is a rescaling of data from the original range to a new range (Min-Max Feature scaling), usually between 0 and 1.

$$x_i' = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

In more complicated cases, normalization may refer to adjustments, where the intention is to bring the entire probability distribution of adjusted values into given alignment



Normalization

```
from sklearn.preprocessing import MinMaxScaler
# example of normalization
# define data, it must be a column vector
data = pd.read_csv('FilRouge.csv',usecols=[1]).values
print(data.T)
# fit transform
transformer = MinMaxScaler()
transformer.fit(data.T)
# normalize transform
transformed = transformer.transform(data)
print(transformed)
# invert normalize
inverted = transformer.inverse_transform(transformed)
print(inverted.T)
```

```
[[128 181  87 219 407 226 214 383 505 387 278 523 572 354 404 673 752 468
  419 725]]
[[0.06165414 0.14135338 0.         0.19849624 0.48120301 0.20902256
  0.19097744 0.44511278 0.62857143 0.45112782 0.28721805 0.6556391
  0.72932331 0.40150376 0.47669173 0.88120301 1.         0.57293233
  0.49924812 0.9593985 ]]
[[128. 181.  87. 219. 407. 226. 214. 383. 505. 387. 278. 523. 572. 354.
  404. 673. 752. 468. 419. 725.]]
```

min = 0
max = 1

Missing values

- There is NO good way to deal with missing data.
- *Imputation* (missing data insertion) does not necessarily give better results.

Imputation algorithms can be divided into simple but fast approaches (like mean imputation) and more advanced algorithms that need more computation time (like Kalman smoothing)

Missing values

Non-time-series specific method

- mean imputation
- median imputation
- mode imputation
- calculate the appropriate measure and replace NAs with the values: this is appropriate for stationary time series.
- random sample imputation
- replace missing values with observations randomly selected from the remaining (either all of of it or just some section): this is not likely to work well unless the random select is carefully chosen.

Missing values

Time-Series specific method

- Last observation carried forward (LOCF)
- Next observation carried backward (NOCB)
- Linear interpolation
- Spline interpolation

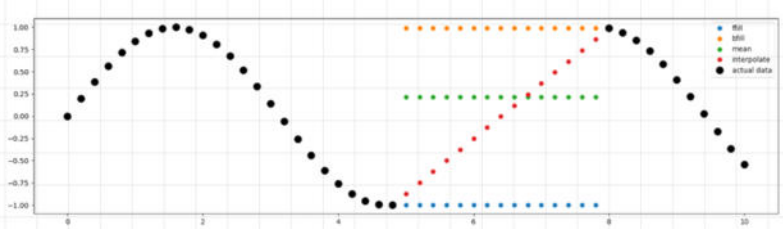
These methods rely on the assumption that adjacent observations are similar to one another.

They do not work well when this assumption is not valid, e.g., when the presence of strong seasonality.

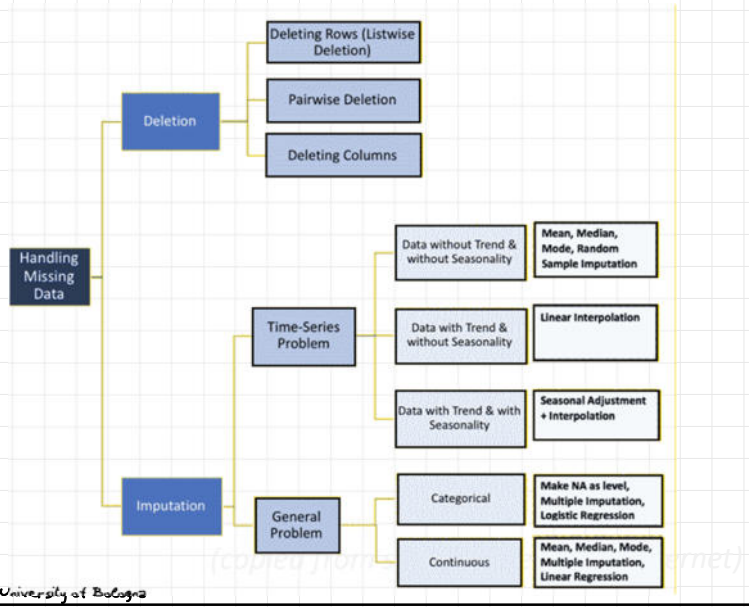
Missing values

Using pandas, simple imputations

```
ds = pd.Series(y) # or ds = pd.read_csv('FilRougeMissing.csv', usecols=[1])
ds1=ds.fillna(method="ffill")
ds2=ds.fillna(method="bfill")
ds3=ds.fillna(ds.mean())
ds4=ds.interpolate()
```



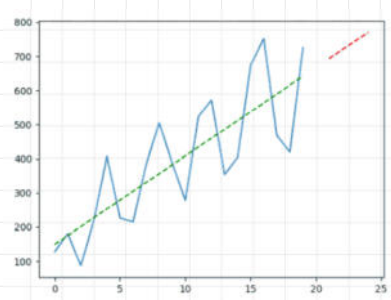
Missing values



Trend line, python

Computing a linear trend (column data, need a transpose)

```
y = pd.read_csv('FilRouge.csv',usecols=[1]).values.flatten()
x = np.arange(len(y))
coeff = np.polyfit(x, y, 1) # here linear (2 for quadratic ...)
# coefficients (slope and intercept)
print(f"Coefficients slope:{coeff[0]} intercept:{coeff[1]}")
pred  = np.polyval(coeff,x) # the linear model
valid = np.polyval(coeff,range(len(x)+1,len(x)+5)) # extension
plt.plot(range(len(x)+1,len(x)+5),valid,"r--")
plt.plot(x, pred, "g--")
plt.plot(y)
plt.show()
```



Forecast accuracy

$X = (x_1, \dots, x_n)$, actual data, $F = (f_1, \dots, f_n)$ forecast data

- BIAS: arithmetic mean of errors $BIAS = \frac{\sum_{i=1}^{n}(x_i - f_i)}{n}$
- MAD (Mean Absolute Deviation): $MAD = \frac{\sum_{i=1}^{n}|x_i - f_i|}{n}$
- MSE (Mean Squared Error) $MSE = \frac{\sum_{i=1}^{n}(x_i - f_i)^2}{n}$
- Standard Error (RMSE): $\mathrm{SE} = \sqrt{MSE}$
- MAPE (Mean Absolute Percent Error) : $MAPE = \frac{\sum_{i=1}^{n}\frac{|x_i - f_i|}{f_i} * 100}{n}$

Python, accuracy metrics

```
import os, numpy as np, pandas as pd
# Accuracy metrics
def forecast_accuracy(forecast, actual):
    mape = np.mean(np.abs(forecast - actual)/np.abs(actual))  # MAPE
    me   = np.mean(forecast - actual)             # ME
    mae  = np.mean(np.abs(forecast - actual))     # MAE
    mpe  = np.mean((forecast - actual)/actual)    # MPE
    rmse = np.mean((forecast - actual)**2)**.5    # RMSE
    corr = np.corrcoef(forecast, actual)[0,1]     # correlation coeff
return({'mape':mape, 'me':me, 'mae': mae, 'mpe': mpe, 'rmse':rmse,
        'corr':corr})
print( forecast_accuracy(forecast_values, test.values) )
```

Preprocessing, exercise

1. Remove blocks of 6, 4 and 3 data from the first 120 of the BoxJenkins dataset
2. Load the modified BoxJenkins data series
3. Reconstruct missing data
4. Separate training, validation and forecast sets ([:-24],[-24,-12],[-12:])
5. Training set, level variance with log or Box-Cox transform
6. Detrend with diff transform, d=1
7. Check for seasonality with ACF plot, find m
8. Deseason with diff transform, d=m
9. Check stationarity with Augmented Dickey Fuller test
10. Check normality of the residuals with Shapiro Wilk test
11. Interpolate a linear regression model on the preprocessed train data
12. Predict and reconstruct validation data and compute accuracies
13. Predict and reconstruct forecast data and compute accuracies
14. Plot original, predicted and forecast data

Advanced topics

The following just sketches of topics that deserve much more grounding and insight.

Consider the content a pointer to issues that could need to be addressed in order to enhance forecasting effectiveness.

Advanced preprocessing

More sophisticated data preprocessing techniques include:

- Outliers detection
- Feature extraction
- Dimensionality reduction
- Noise reduction
- Data augmentation

Outliers

Also known as anomaly detection.

An outlier is an observation that significantly differs from other observations of the same feature.

Time series with outliers have several characteristics:

- There is a systematic pattern (which is deterministic) and some variation (which is stochastic).
- Only a few data points are outliers.
- Outliers are significantly different from the rest of the data.

An anomaly detection algorithm must either:
- label each time point with *anomaly/not anomaly*,
- or forecast a signal for some point and test if this point value varies from the forecasted enough to deem it as an anomaly.

Outliers

Possible decomposition approaches:

- STL decomposition: seasonal-trend decomposition procedure.

  This technique gives you an ability to split your time series signal into three parts: seasonal, trend and residue.

  If you analyze deviation of residue and introduce some threshold for it, you'll get an anomaly detection algorithm. See forward.

  Similarly with ARIMA or exponential smoothing

- Classification and regression trees. use supervised learning to teach trees to classify anomaly and non-anomaly data points.

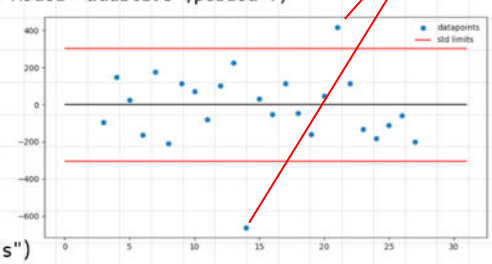
Caution: in certain applications outliers are data of special interest, not data to be deleted.

Outliers, via STL decompose

```
from statsmodels.tsa.seasonal import seasonal_decompose

df=pd.read_csv("traffico16.csv")
col    = "ago2"
series = df[col].fillna(value=df[col].mean())
result = seasonal_decompose(series, model='additive',period=7)
Observed = result.observed
Trend    = result.trend
Seasonal = result.seasonal
Resid    = result.resid
std = resid.std()
```

Outliers




```
plt.figure(figsize=(10,5))
plt.plot(Resid,"o",label="datapoints")
plt.hlines(0,0,len(resid))
plt.hlines(1.5*std,0,len(Resid),color="red",label="std limits")
plt.hlines(-1.5*std,0,len(Resid),color="red")
plt.legend()
plt.show()
```

Dimensionality reduction

In case of multivariate analysis, datasets can be defined over too many dimensions.

Attributes can number up to the hundreds or even the thousands.

It can be useful / necessary to reduce the number of attributes:

- Space required to store the data is reduced
- Less dimensions lead to less computation/training time
- Some algorithms do not perform well when on large dimensions
- It takes care of multicollinearity by removing redundant features.
- It helps in visualizing data.

Dimensionality reduction

Dimensionality reduction can be done in two different ways:

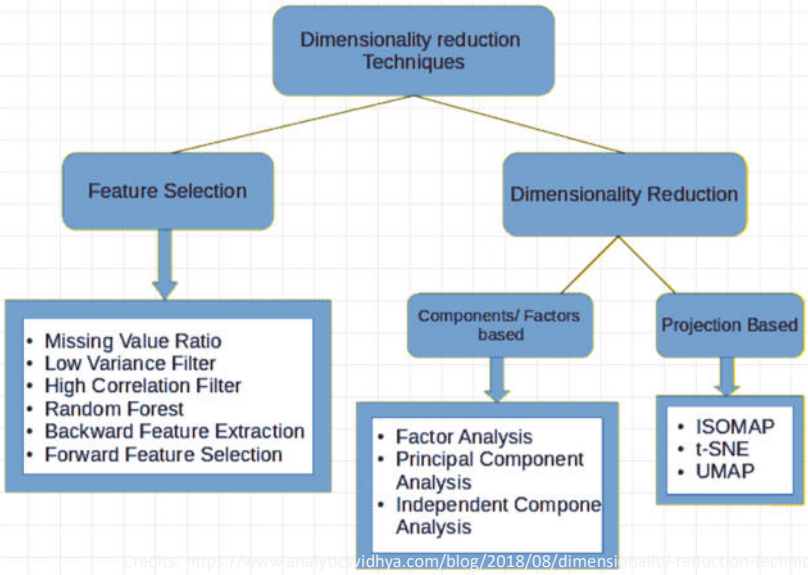
1. By only keeping the most relevant variables from the original dataset (this technique is properly called feature selection)
2. By finding a smaller set of new variables, each being a combination of the input variables, containing basically the same information as the input variables (this technique is sometimes called dimensionality reduction itself)

Dimensionality reduction

Some techniques for dimensionality reduction:

- Missing Value Ratio: attributes with too many missing values (50%?) can be discarded.
- Low Variance Filter: attributes often repeating one same value can be discarded
- High Correlation filter: if two variables are highly correlated, one can be dropped (keeping the one most correlated with the target variable)
- Random Forest: very popular algorithm
- Backward Feature Elimination: try to re-train the model omitting one feature. If its removal gives no big effect, it can be removed.
- Forward Feature Selection: start with few attributes and include in the model the features which contribute a greater improvement.
- Principal Component Analysis (PCA): very popular, a technique extracting a new set of variables from an existing large set of variables by linear combinations. These newly extracted variables are called Principal Components.
- Singular Value Decomposition (SVD) another mathematical approach to decompose the original dataset into its constituents.

Dimensionality reduction

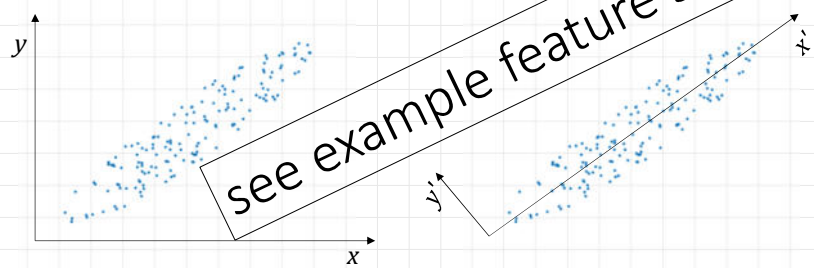


Credits: https://www.analyticsvidhya.com/blog/2018/08/dimensionality-reduction-techniques-python/

PCA

Principal Component Analysis (PCA) performs a *transformation of the coordinate system* into components, and the principal components are chosen such that they maximally explain the variance in the data vectors. The principal components can be found by calculating the eigenvectors of the covariance matrix of the data.)

```
from sklearn.decomposition import PCA
sk_model = PCA(n_components=10)
sk_model.fit_transform(features_ndarray)
print(sk_model.explained_variance_ratio_.cumsum())
```



see example feature selection

Feature extraction

Principal components correspond to new descriptive features computed based on the input ones, this is feature extraction.

Feature extraction creates new features, typically less than the original ones, from an initial set of data with the objective of easing machine learning.

These features encapsulate the central properties of a data set and represent it in a low-dimensional space that facilitates learning.

Feature extraction is used to provide a more manageable, representative subset of input variables. These new reduced set of features should then be able to summarize most of the information contained in the original set of features.

Data augmentation

Data augmentation (relevant for machine learning) means generating artificial data in order to reduce the variance of the output (to reduce the number of errors). Widely used in image analysis, but useful also when dealing with too short time series.

Some of the common techniques used in data augmentation, especially for time series classification, include:

1. Extrapolation: the relevant fields are filled with values based on heuristics. For example a trend-seasonality model can compute predicted values for time instants not covered by the series.
2. Tagging: common records are tagged to a group.
3. Aggregation: values are estimated using mathematical values such as averages and means
4. Probability: values are populated based on the probability of events.

See, f.e., *https://tsaug.readthedocs.io/en/stable/*

Noise reduction

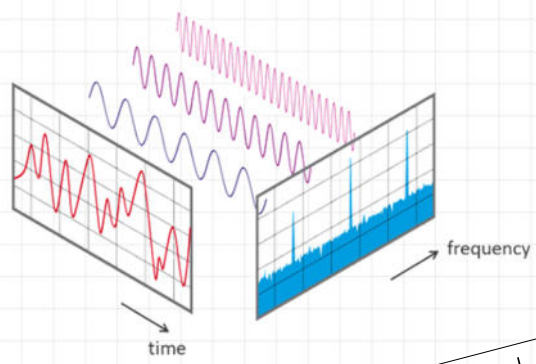
Denoising, can be heavily borrowed from signal processing techniques to improve the *signal-to-noise ratio (SNR)*.

Analysis in the frequency domain: apply to the series Fourier transform (or wavelet transform) and then an appropriate filter. We see 3 cases: low/high pass, autoencoder, Kalman filter.

- *Low pass filter:* It passes signals with a frequency lower than a certain cut-off frequency and attenuates signals with frequencies higher than that. In time series, a *simple moving average (SMA)* is a low pass filter.
- *High pass filter:* It passes signals with a frequency higher than a certain cut-off frequency and attenuates signals with frequencies lower than that. For time series, either linear filters (such as SMA) or non-linear filters (such as median filter) can be used. Consider also Kalman filter.

Spectral analysis

Any periodic function (or function defined on an interval) can be reconstructed by adding an appropriate (infinite) number of sinusoids with appropriate amplitudes and frequencies.



see spectral_example.py

Spectral analysis, Fourier

Spectral analysis is a technique to discover periodicities underlying time series. To his end, we first transform data from time to frequency domain.

The mathematics is complex, it is based on Fourier transforms, that permit to represent any continuous function $f(t)$ as a weighted sum of sinusoidal functions.

The general formula is $\hat{f}(t) = \int_{-\infty}^{\infty} f(t)e^{-2\pi i t\xi}dt$, for any real number ξ. The transformation is invertible, thus from $\hat{f}(t)$ we can come back to $f(t)$. The Fourier transform of a function of time is a complex-valued function of frequency, whose magnitude represents the amount of that frequency present in the original function,

For data series, the Discrete Fourier Transform (DFT) converts a finite sequence of equally-spaced samples of a function into a same-length sequence of equally-spaced samples of a complex-valued function of frequency (the discrete-time Fourier transform, DTFT).

The Fast Fourier Transform (FFT) is an algorithm that efficiently computes the DFT of a sequence (O(n lg n)).

Spectral analysis, FFT

FFT: the transform of a sequence of real values, shows a symmetry between the k-th and the n-k-th elements, with k = 0, 1, … , n: $Re\big(f(k)\big) = Re(f(n-k))$ (symmetry around n/2): only half of the transform data is needed.

The Nyquist(-Shannon) sampling theorem states that a signal containing frequency components up to frequency F must sampled at a rate at least 2F in order to be fully reconstructed. Conversely, sampling at frequency F can reconstruct the signal only up to the F/2 frequency component (its Nyquist frequency).

Noise has high frequency. Cutting out high frequencies prevents from representing noise.

Spectral analysis: low pass

Better a stationary signal, need to detrend a series and keep only the seasonal component

- `seasonal = seasonal_decompose(...).seasonal` or
- `scipy.signal.detrend()`

Only half of the transformed values are used (because of fft symmetry).

Periods (wavelengths) are the inverse of frequencies.

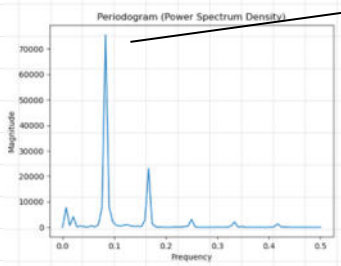
Spectral analysis

```
detrended_series = signal.detrend(numbers,type="linear")
fft = np.fft.fft(detrended_series)
one_sided_fft = fft[:int(n/2)+1]
# magnitude is power of 2 divided by length
power = np.abs(one_sided_fft)**2 / n
```

see stl.py

Note: consider also `np.fft.rfft`, `np.fft.irfft`, …

Ex. For airline passengers:



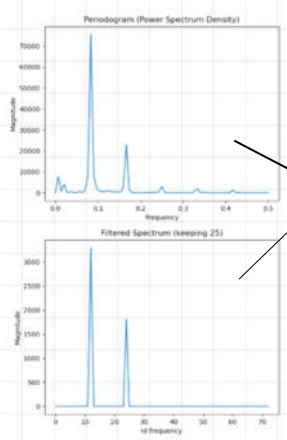
Max power at freq. 0.0833
Period is 1/0.0833 = 12

see spectral.py

Low pass denoising

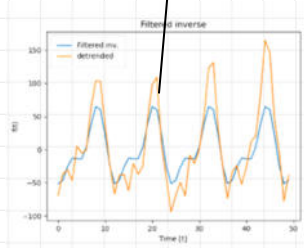
Noise is reduced by keeping only the most relevant frequencies and discarding the highest ones.

Ex. Airline passengers:



Original and filtered spectra

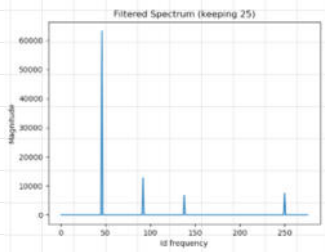
Original and reconstructed signal

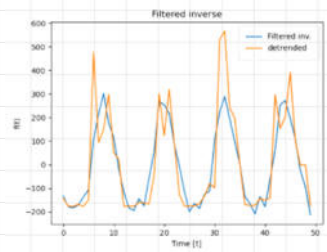


Low pass denoising

Ex.: rainfall dataseries.

Filtered spectrum and original / low pass reconstruction signals.





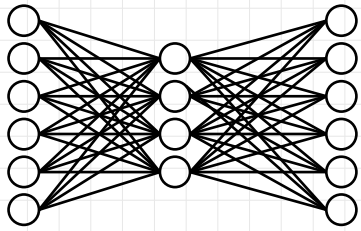
Noise reduction: Autoencoders

A neural approach to noise filtering: autoencoders.

An autoencoder is a simple neural network trained with the same dataset both as input and as output of the network, where the network has fewer hidden neurons than the input / output ones.

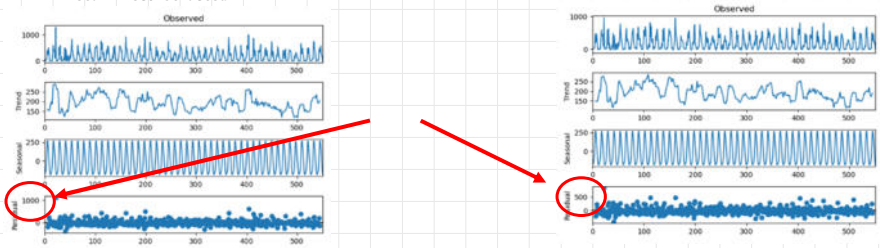
Since noise has much higher dimensions than regular data, this process reduces the noise.

This is similar to Principal Component Analysis (PCA) and is a general dimensionality reduction technique.



Autoenconders

```
class Autoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super(Autoencoder, self).__init__()
        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 64), nn.ReLU(),
            nn.Linear(64, latent_dim),nn.ReLU()
        )
        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64),nn.ReLU(),
            nn.Linear(64, input_dim), nn.Sigmoid()  # Use Sigmoid to ensure output is in [0, 1]
        )
    def forward(self, x):
        latent = self.encoder(x)
        reconstructed = self.decoder(latent)
        return reconstructed
```



Noise reduction: Kalman filters

Kalman filter is an algorithm, very widely used in different applications, that provides estimates of some unknown variables given their measurements observed over time. Filter because it filters out noise. If all noise is Gaussian, the Kalman filter minimises the mean square error of the estimated parameters. If the noise is NOT Gaussian the Kalman filter is the best linear estimator but non-linear estimators may be better.

Lots of literature, here only a shallow sketch.

Kalman filters have relatively simple form and require small computational power (they don't need to keep any history other than the previous state), and they are very fast, making them well suited for real time problems and embedded systems.

They can deal with noise, assuming that errors have a Gaussian distribution, in which case they are mean squared error minimizers.

We work on univariate (1D) time series, but they are defined also for multivariate.

Kalman filters

The Kalman filter algorithm consists of two stages: prediction and update (note that the terms “prediction” and “update” are often called “propagation” and “correction,” respectively, in different literature).

1. Prediction phase: the filter *attempts to predict the next value* in the series based on the current value of the series and its parameters.
2. Update phase: the filter *reads the actual measurement* and it updates its parameters and corrects its prediction.

These two steps are repeated cyclically for each new measurement. Once you have the filtered series, you can make forecasts by training the model you have chosen using the "clean" data. In this way you will get a more accurate forecast.

(see also https://towardsdatascience.com/kalman-filter-an-algorithm-for-making-sense-from-the-insights-of-various-sensors-fused-together-ddf67597f35e)

Kalman filter predictions

The Kalman filter model assumes the true state at time t, $\mathbf{x}_t$, is evolved from the state at $(t-1)$ according to $\mathbf{x}_t = \mathbf{F}_t \mathbf{x}_{t-1} + \mathbf{B}_t \mathbf{u}_t + \mathbf{w}_t$ where

- $\mathbf{F}_t$ is the state transition model which is applied to the state $\mathbf{x}_{t-1}$;
- $\mathbf{B}_t$ is the control-input model which is applied to the input vector $\mathbf{u}_t$;
- $\mathbf{w}_t$ is the process noise, which is assumed to be drawn from a zero mean multivariate normal distribution .

At time t an observation $\mathbf{z}_t$ of the state $\mathbf{x}_t$ is made as $\mathbf{z}_t = \mathbf{H}_t \mathbf{x}_t + \mathbf{v}_t$ where

- $\mathbf{H}_t$ is the observation model, which maps the true state space into the observed space and
- $\mathbf{v}_t$ is the observation noise, which is assumed to be zero mean Gaussian white noise

Matrices need to be identified for the specific application.

Kalman filters

Predict: In this step the Kalman filter predicts the new values from the initial value and then predicts the uncertainty/error/variance in the prediction according to the process noise present in the system.

Update: In this step the filter takes the actual measured value from the devices of the system.

The filter then computes the difference between the predicted value and the measured value and decides which value to keep (both predicted value and measured value are unreliable) by calculating the *Kalman Gain*. Based upon the decision a new value and new uncertainty/ error/ variance are predicted.

Kalman Gain K determines whether the predicted or the measured value is closer to the actual value. Its value ranges from 0 to 1: if near to 0 the predicted value is close to the actual value, if near to 1 the measured value is close to the actual value.

K = Error In Prediction / (Error in Prediction + Error in Measurement)

The **output** from the update step is again fed back to the prediction step and the process continues till the difference between the predicted value and the measured value converges to zero.

Kalman filters

Actual measure
(can be state initialization)

Update:
1. Compute Kalman gains
2. Update the estimate with measurements
3. Update the error covariance.

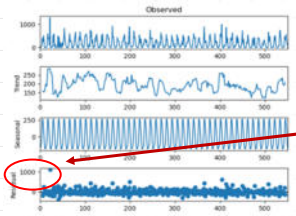
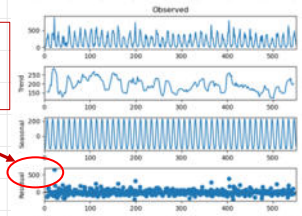
Predict:
1. Predict next state
2. Predict next error covariance.

Kalman filters

https://pykalman.github.io/

```
from pykalman import KalmanFilter
...
measurements = y.to_numpy()
kf = KalmanFilter(transition_matrices=[1],
observation_matrices=[1],
initial_state_mean=measurements[0],
initial_state_covariance=1,
observation_covariance=10,
transition_covariance=10)
state_means, state_covariances = kf.filter(measurements)
```

Parameters.
Make sense after studying.



Randomness reduced by 50%!